Q1. Signed binary arithmetic using 8-bit 2's complement

The signed 8-bit range is −128 to +127.

(a) Calculate 25 + (+18)

Step 1: Convert 25 to 8-bit binary. 25 = 16 + 8 + 1, therefore 25 = 00011001.

Step 2: Convert +18 to 8-bit binary. 18 = 16 + 2, therefore +18 = 00010010.

Step 3: Add the binary numbers.

```
  00011001   (25)
+ 00010010   (+18)
-----------
  00101011
```

Step 4: Convert the result to decimal. 00101011 = 32 + 8 + 2 + 1 = 43.

Step 5: Check overflow. 43 is within −128 to +127, so overflow does not occur.

Final answer: Binary = 00101011; Decimal = +43; Overflow = No.

(b) Calculate (−25) + (−18)

Step 1: Convert +25 to binary and find its 2's complement.

```
+25:       00011001
Invert:    11100110
Add 1:   + 00000001
           --------
−25:       11100111
```

Step 2: Convert +18 to binary and find its 2's complement.

```
+18:       00010010
Invert:    11101101
Add 1:   + 00000001
           --------
−18:       11101110
```

Step 3: Add the binary numbers.

```
  11100111   (−25)
+ 11101110   (−18)
-----------
1 11010101
```

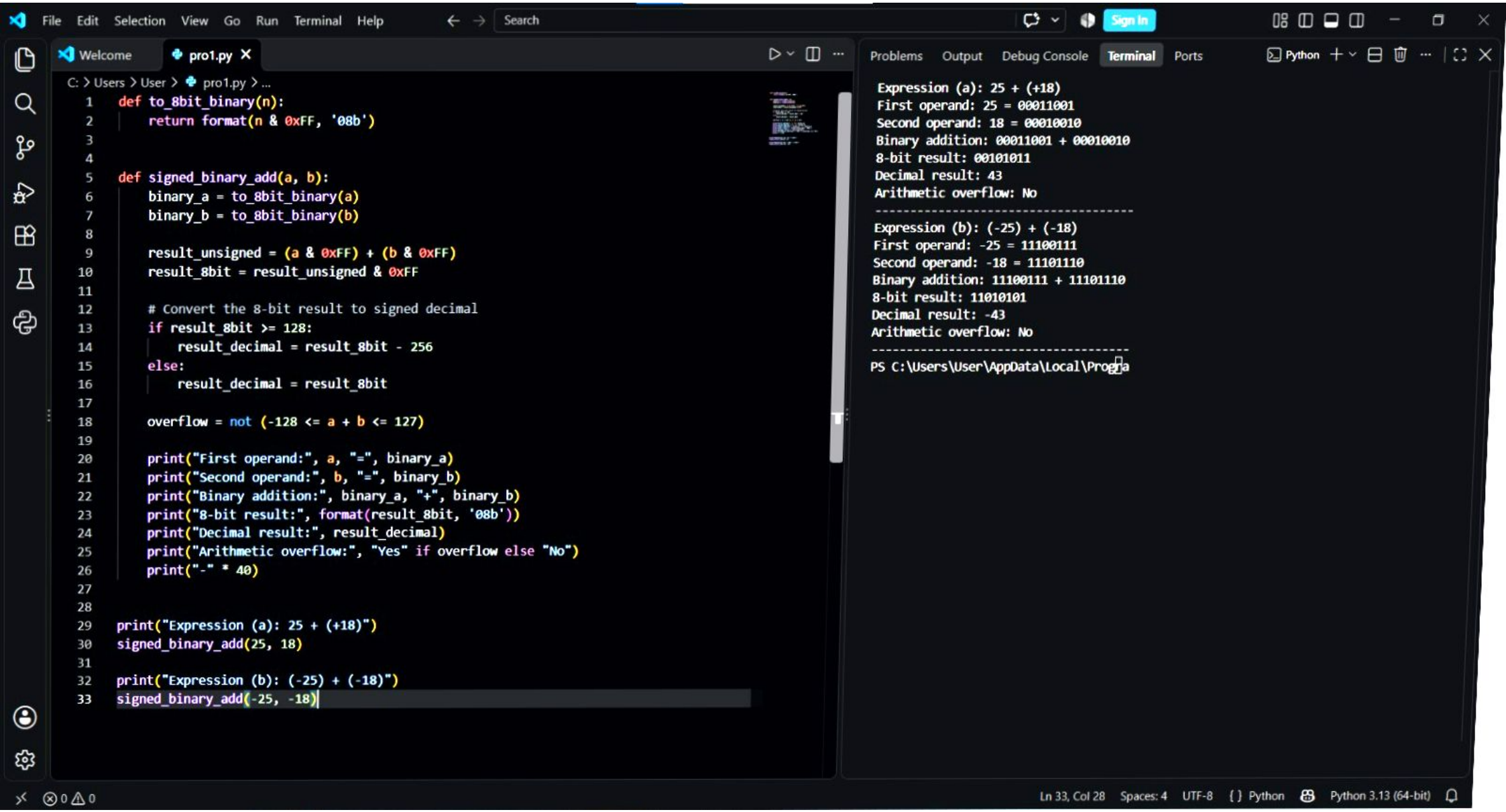
Discard the carry beyond the 8th bit. The 8-bit result is 11010101.

Step 4: Convert the result to decimal. The most significant bit is 1, so the number is negative.

```
Invert:    11010101 → 00101010
Add 1:     00101010 + 1 = 00101011
Magnitude: 32 + 8 + 2 + 1 = 43
Result:    −43
```

Step 5: Check overflow. −43 is within −128 to +127, so overflow does not occur.

Final answer: Binary = 11010101; Decimal = −43; Overflow = No.


File Edit Selection View Go Run Terminal Help
Search
Sign in
Welcome
pro1.py
C: > Users > User > pro1.py > ...
def to_8bit_binary(n):
return format(n & 0xFF, '08b')
def signed_binary_add(a, b):
binary_a = to_8bit_binary(a)
binary_b = to_8bit_binary(b)
result_unsigned = (a & 0xFF) + (b & 0xFF)
result_8bit = result_unsigned & 0xFF
Convert the 8-bit result to signed decimal
if result_8bit >= 128:
result_decimal = result_8bit - 256
else:
result_decimal = result_8bit
overflow = not (-128 <= a + b <= 127)
print("First operand:", a, "=", binary_a)
print("Second operand:", b, "=", binary_b)
print("Binary addition:", binary_a, "+", binary_b)
print("8-bit result:", format(result_8bit, '08b'))
print("Decimal result:", result_decimal)
print("Arithmetic overflow:", "Yes" if overflow else "No")
print("-" * 40)
print("Expression (a): 25 + (+18)")
signed_binary_add(25, 18)
print("Expression (b): (-25) + (-18)")
signed_binary_add(-25, -18)
Problems Output Debug Console Terminal Ports
Python
Expression (a): 25 + (+18)
First operand: 25 = 00011001
Second operand: 18 = 00010010
Binary addition: 00011001 + 00010010
8-bit result: 00101011
Decimal result: 43
Arithmetic overflow: No
Expression (b): (-25) + (-18)
First operand: -25 = 11100111
Second operand: -18 = 11101110
Binary addition: 11100111 + 11101110
8-bit result: 11010101
Decimal result: -43
Arithmetic overflow: No
PS C:\Users\User\AppData\Local\Progra
Ln 33, Col 28 Spaces: 4 UTF-8 { } Python Python 3.13 (64-bit)

Q2. Logical shift operations on 8-bit binary numbers

(a) Logical left shift of 10110110 by 2 positions

Rule: Shift bits left, discard the leftmost bit, and insert 0 on the right.

Step 1: Original number

```
10110110
```

Step 2: First left shift

```
10110110 → 01101100
```

Step 3: Second left shift

```
01101100 → 11011000
```

Step 4: Final result

```
Final 8-bit result: 11011000
```

(b) Logical right shift of 11010100 by 2 positions

Rule: Shift bits right, discard the rightmost bit, and insert 0 on the left.

Step 1: Original number

```
11010100
```

Step 2: First right shift

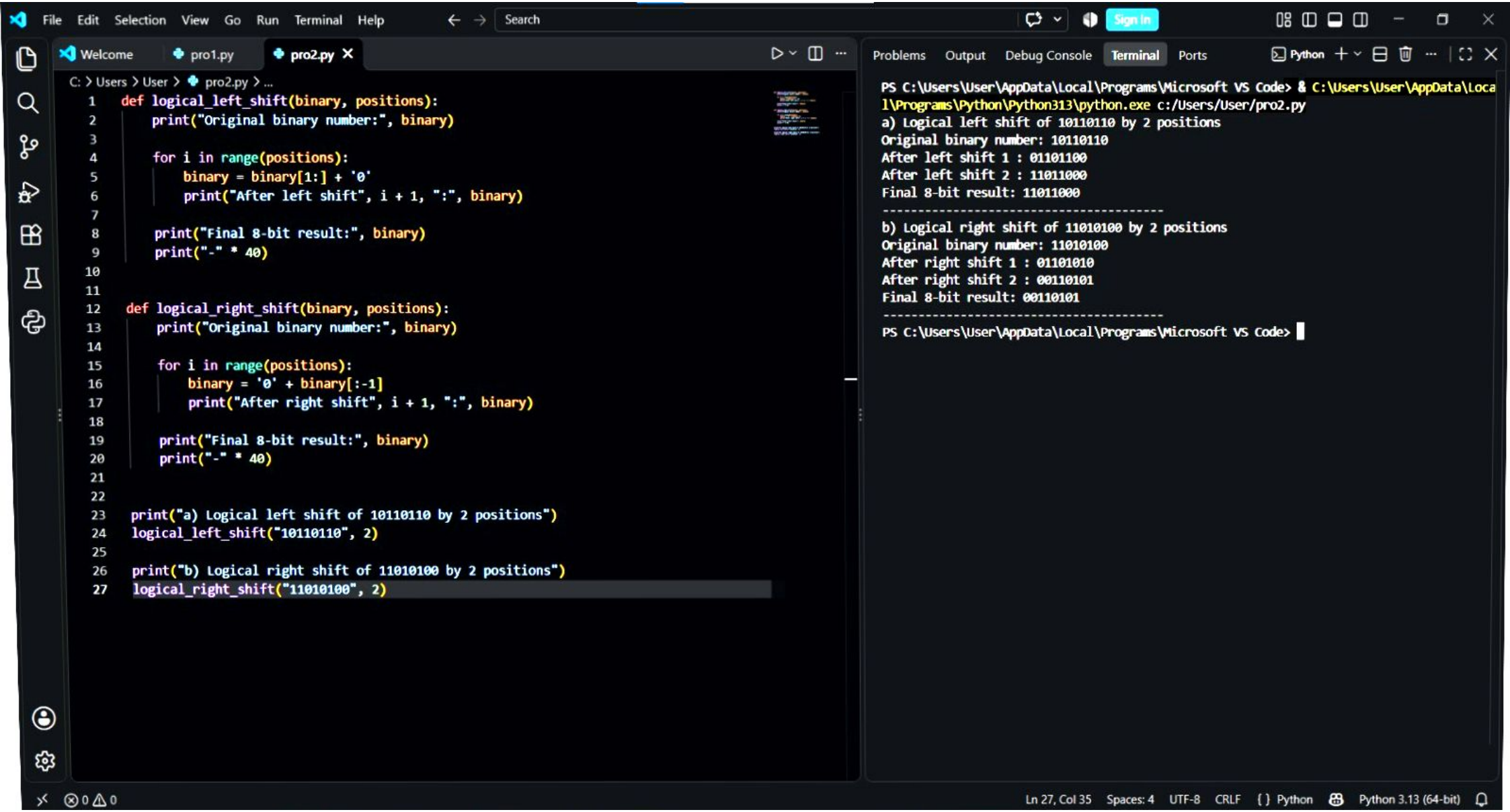
```
11010100 → 01101010
```

Step 3: Second right shift

```
01101010 → 00110101
```

Step 4: Final result

```
Final 8-bit result: 00110101
```


File Edit Selection View Go Run Terminal Help
Search
Sign In
Welcome
pro1.py
pro2.py
C: > Users > User > pro2.py > ...
def logical_left_shift(binary, positions):
print("Original binary number:", binary)
for i in range(positions):
binary = binary[1:] + '0'
print("After left shift", i + 1, ":", binary)
print("Final 8-bit result:", binary)
print("-" * 40)
def logical_right_shift(binary, positions):
print("Original binary number:", binary)
for i in range(positions):
binary = '0' + binary[:-1]
print("After right shift", i + 1, ":", binary)
print("Final 8-bit result:", binary)
print("-" * 40)
print("a) Logical left shift of 10110110 by 2 positions")
logical_left_shift("10110110", 2)
print("b) Logical right shift of 11010100 by 2 positions")
logical_right_shift("11010100", 2)
Problems
Output
Debug Console
Terminal
Ports
Python
PS C:\Users\User\AppData\Local\Programs\Microsoft VS Code> & C:\Users\User\AppData\Local\Programs\Python\Python313\python.exe c:/Users/User/pro2.py
a) Logical left shift of 10110110 by 2 positions
Original binary number: 10110110
After left shift 1 : 01101100
After left shift 2 : 11011000
Final 8-bit result: 11011000
b) Logical right shift of 11010100 by 2 positions
Original binary number: 11010100
After right shift 1 : 01101010
After right shift 2 : 00110101
Final 8-bit result: 00110101
PS C:\Users\User\AppData\Local\Programs\Microsoft VS Code>
Ln 27, Col 35
Spaces: 4
UTF-8
CRLF
Python
Python 3.13 (64-bit)